



GRAFİK PROGRAMLAMA DERSİ VİZE ÖDEVİ

Basit 2D Fizik Motoru Geliştirme: Çarpışma, Constraint ve Stabilite Analizi

Öğrenci Adı Soyadı: Polat bilir

Öğrenci Numarası: 2301060020

Bölüm: Yazılım Mühendisliği

Teslim Tarihi: 03.05.2026

2. Amaç

Bu projenin amacı, temel fizik motoru bileşenlerini iki boyutlu (2B) bir simülasyon ortamında sıfırdan geliştirerek, gerçek zamanlı fizik tabanlı animasyon ve etkileşim mantığını kavramaktır. Geliştirilen mini fizik motoru kapsamında; parçacık ve rijit cisim temsili, yerçekimi etkisi, farklı sayısal integrasyon yöntemlerinin uygulanması, çarpışma tespiti (collision detection) ve impuls tabanlı tepkisi ile mesafe kısıtlarının (distance constraints) çözülmesi hedeflenmiştir. Ek olarak, yalnızca görsel bir animasyon üretmekle kalınmamış, Semi-Implicit Euler ve Velocity Verlet integrasyon yöntemlerinin enerji korunumu ve kararlılık (stabilite) açısından karşılaştırmalı analizi yapılarak sistem davranışı incelenmiştir.

3. Yöntem ve Kullanılan Denklemler

Bu bölümde, simülasyonun temelini oluşturan matematiksel modeller ve hesaplamalı algoritmalar sunulmaktadır.

3.1. Temel Dinamik Model

Simülasyondaki her bir cismin hareketi, klasik Newton mekaniği temel alınarak modellenmiştir. Cisme etki eden toplam kuvvet (F), kütle (m) ve ivme (a) ilişkisi Newton'un İkinci Yasası ile tanımlanmıştır:

$$F = m \cdot a$$

Simülasyonda sabit dış kuvvet olarak yalnızca yerçekimi vektörü ($g = (0, -9.81)$) sisteme dahil edilmiştir. Sayısal işlemlerde sonsuz kütleli (zemin, bağlantı noktası gibi) statik cisimleri modelleyebilmek ve sıfıra bölme hatalarını engellemek amacıyla doğrudan kütle yerine ters kütle (inverse mass, $1/m$) yaklaşımı kullanılmıştır.

3.2. Sayısal İntegrasyon Yöntemleri

Diferansiyel hareket denklemleri ayırık zaman adımlarında (Δt) iki farklı yöntemle çözümlenerek karşılaştırılmıştır:

Semi-Implicit (Symplectic) Euler: Hızın önce güncellenip, konumun bu yeni hız değeriyle güncellendiği birinci mertebeden bir yöntemdir. Basit olmasına karşın sisteme yapay enerji ekleme eğilimindedir.

$$v_{t+1} = v_t + a_t \cdot \Delta t$$

$$x_{t+1} = x_t + v_{t+1} \cdot \Delta t$$

Velocity Verlet: Hem konum hem de hızın güncellenmesini sağlayan, zaman tersinirliğine sahip daha gelişmiş bir yaklaşımdır. Enerji korunumu açısından Euler yöntemine kıyasla daha kararlı sonuçlar üretir ve uzun süreli simülasyonlarda enerji drifti (sapması) çok daha düşüktür.

$$x_{t+1} = x_t + v_t \cdot \Delta t + 0.5 \cdot a_t \cdot \Delta t^2$$

$$v_{t+1} = v_t + 0.5 \cdot a_t \cdot \Delta t$$

Not: Motorumuzda dış kuvvetler sabit olduğu için $a_t = a_{t+1}$ kabul edilerek basitleştirilmiş form kullanılmıştır.

3.3. Çarpışma Tespiti ve Tepkisi

Çarpışma anında oluşan ani hız değişimi, momentum korunumuna dayanan İmpuls (J) yaklaşımı ile çözülmüştür. İki cismin birbirinin içine girmesini (penetrasyon) önlemek için önce kütlelerle ters orantılı bir pozisyon düzeltmesi yapılmış, ardından impuls uygulanmıştır. İmpuls büyüklüğü (j), bağıl hız (v_{rel}), çarpışma normali (n) ve sekme katsayısı (e) cinsinden aşağıdaki gibi hesaplanmıştır:

$$j = \frac{-(1 + e) \cdot (v_{rel} \cdot n)}{\frac{1}{m_1} + \frac{1}{m_2}}$$

3.4. Mesafe Kısıtı (Distance Constraint)

Sarkaç sistemini oluşturmak için cisimler arasına mesafe kısıtı eklenmiştir. Kısıt çözücü (Constraint Solver), cisimler arasındaki anlık mesafeyi olması gereken orijinal mesafe ile kıyaslar. Aradaki hata payı, iteratif bir yaklaşımla cisimlerin ters kütlelerine oranlanarak pozisyonlarına doğrudan düzeltme (projection) olarak yansıtılır.

4. Program Mimarisi

Geliştirilen fizik motoru, modülerliğin sağlanması ve sistemin farklı bileşenlerinin birbirinden izole edilmesi amacıyla Nesne Yönelimli Programlama (OOP) prensipleriyle Python dilinde tasarlanmıştır. Mimarinin temel sınıfları ve görevleri şu şekildedir:

Body Sınıfı: Sistemdeki fiziksel varlıkları (parçacık/daire) temsil eder. Her cismin konum, hız, ivme, kütle, yarıçap, sekme katsayısı (restitution) ve statik/dinamik olma durumu bu sınıfta tutulur. Sonsuz kütleli nesneler (zemin veya bağlantı noktaları) için ters kütle (inverse mass) değeri sıfır olarak atanmıştır.

DistanceConstraint Sınıfı: İki cisim arasındaki mesafenin korunmasını sağlayan kısıt çözücüdür. Cisimler arasındaki anlık hata payını (violation) hesaplar ve kütle merkezlerine göre pozisyon düzeltmesi uygular.

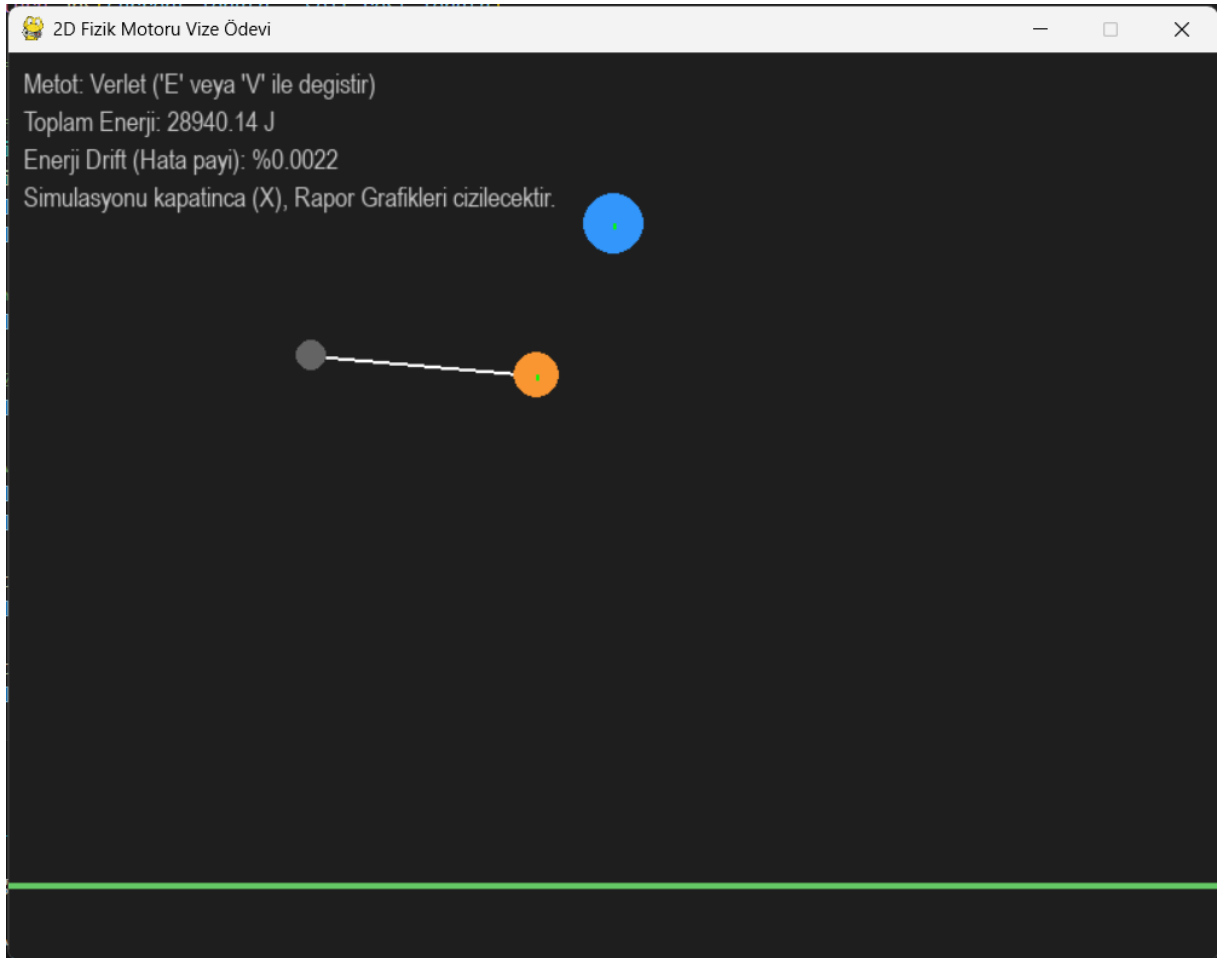
PhysicsWorld Sınıfı: Simülasyonun ana boru hattıdır (pipeline). Her bir zaman adımında (Δt) sırasıyla kuvvetlerin uygulanması, sayısal integrasyon, kısıtların iteratif çözümü ve çarpışma tespiti/tepkisi işlemlerini yürütür.

Ana Döngü ve Görselleştirme: Gerçek zamanlı grafik çizimi için Pygame kütüphanesi, toplanan verilerin simülasyon sonunda loglanıp grafik haline getirilmesi için ise Matplotlib kullanılmıştır.

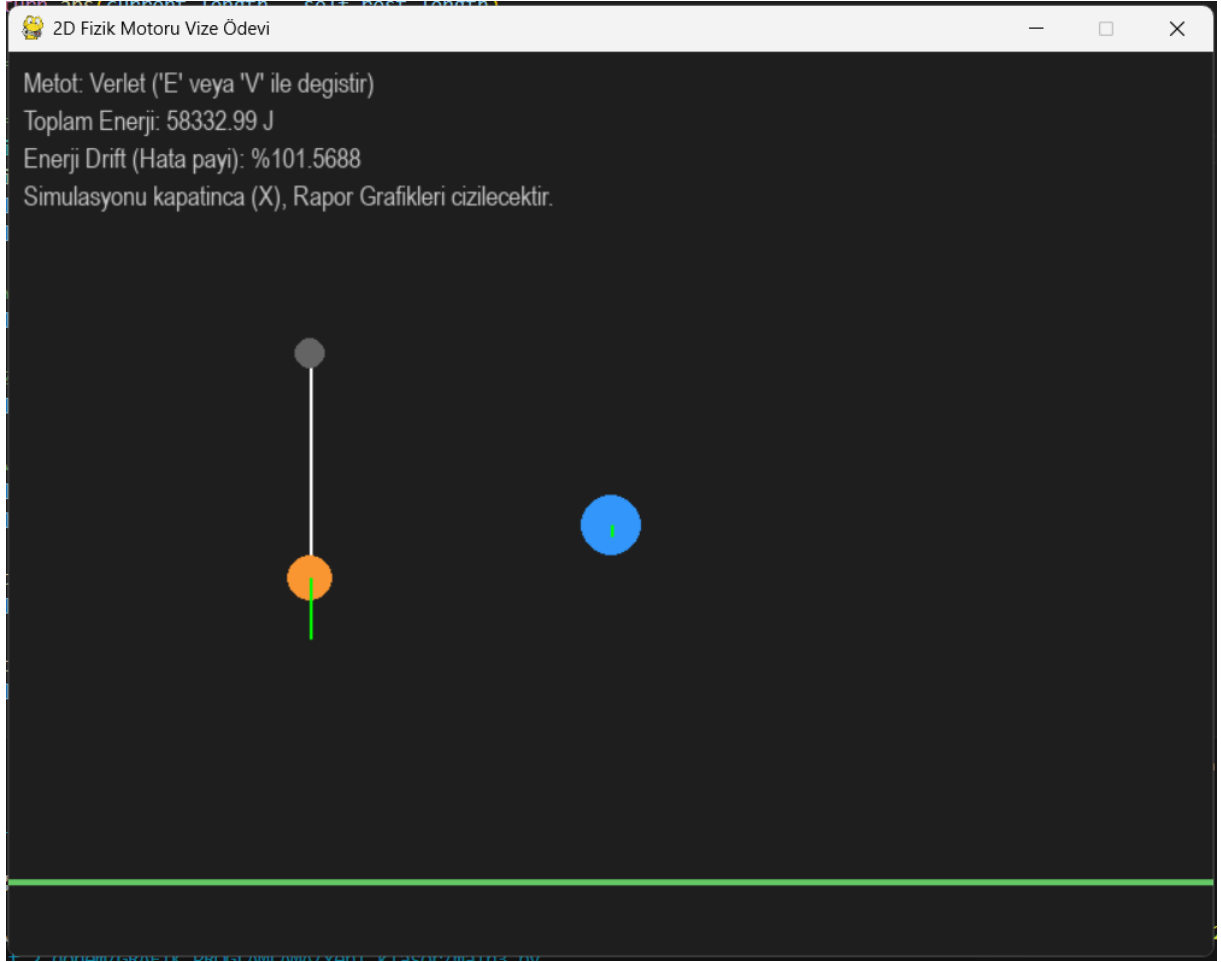
5. Deney Senaryoları ve Sonuç Grafikleri

Sistemin kararlılığını test etmek için bir deney senaryosu oluşturulmuştur. Sahnede aynı anda serbest düşüş yapan bir cisim ve uzaysal bir statik noktaya (anchor) mesafe kısıtıyla bağlı bir sarkaç (pendulum) bulunmaktadır. Cisimler yerçekimi etkisi altında ivmelenmiş, zeminle ve birbirleriyle çarpışmışlardır. Simülasyon sırasında hem Semi-Implicit Euler hem de Velocity Verlet metotları kullanılarak sistemin toplam enerjisi kaydedilmiştir.

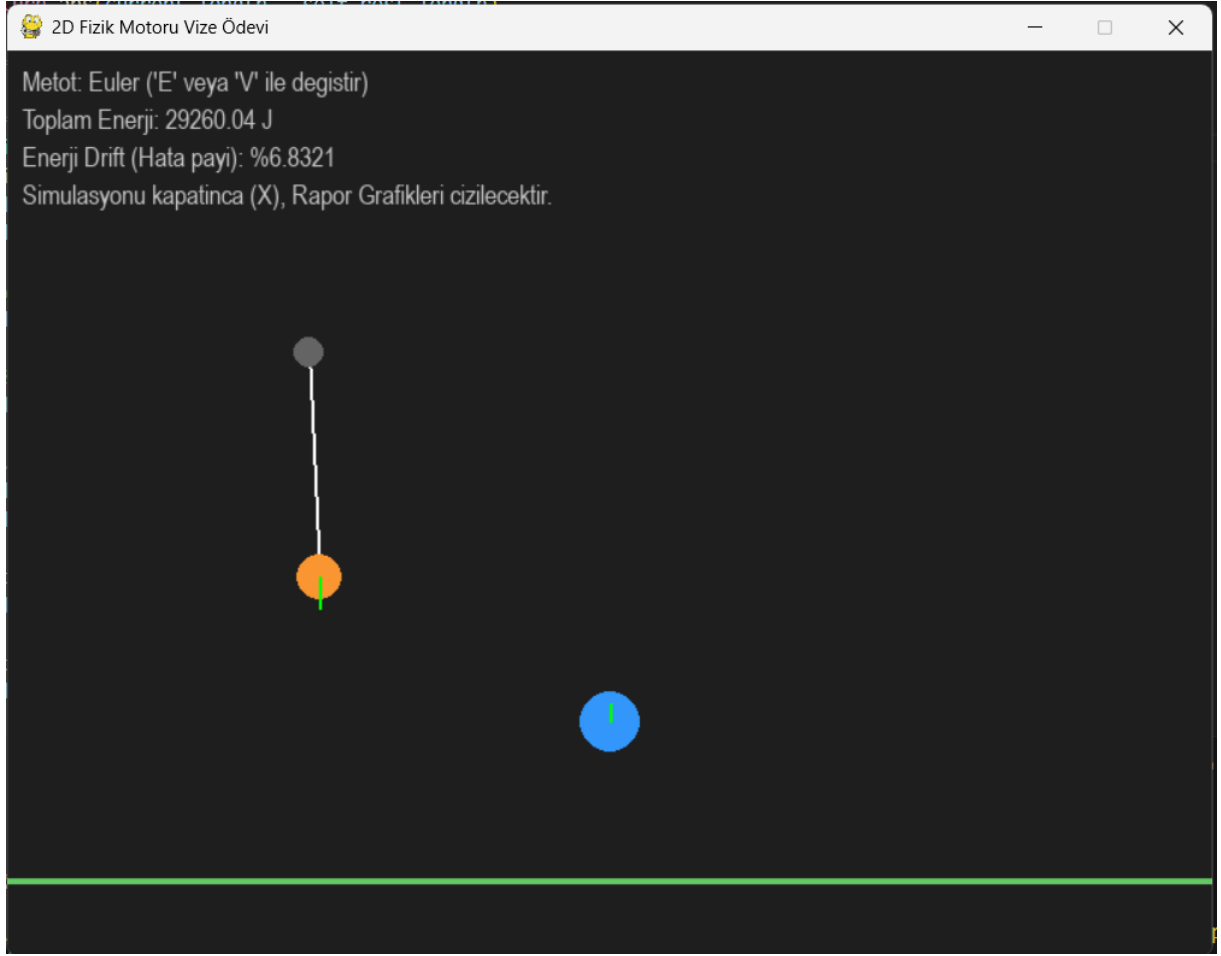
5.1. Simülasyon Görselleri



Şekil 1: Velocity Verlet integrasyon yöntemi kullanılarak çalıştırılan 2B fizik simülasyonunun anlık ekran görüntüsü. Serbest düşen cisim, sarkaç kısıtı (beyaz çizgi) ve anlık hız vektörleri (yeşil çizgiler) görülmektedir.



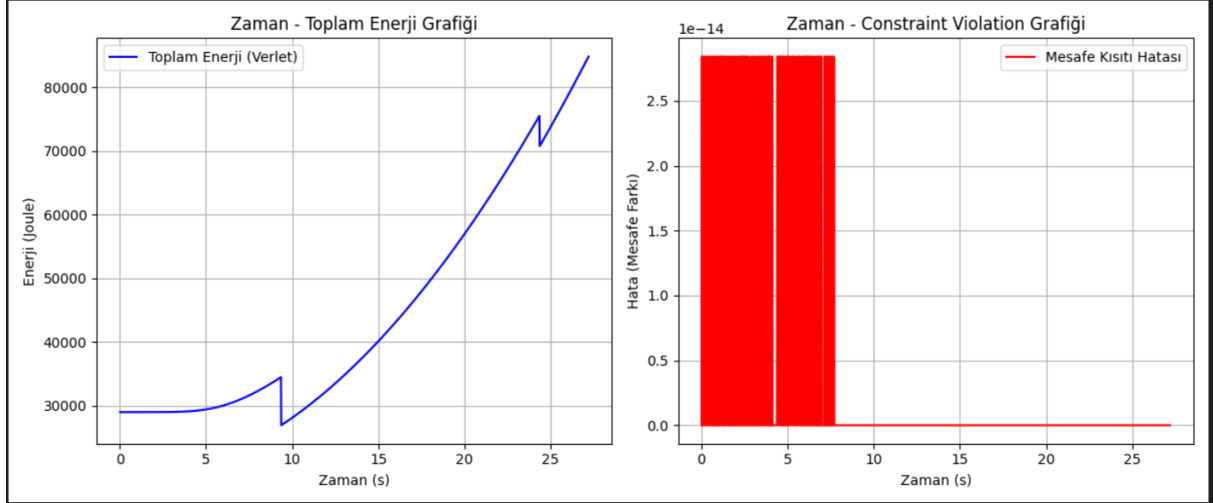
Şekil 2: Simülasyonun ilerleyen saniyelerinde sarkaç sisteminin aşağı yönlü salınımı. Cisimlerin kütle merkezlerinden çıkan yeşil oklar, anlık hız vektörlerinin yönünü ve şiddetini göstermektedir.



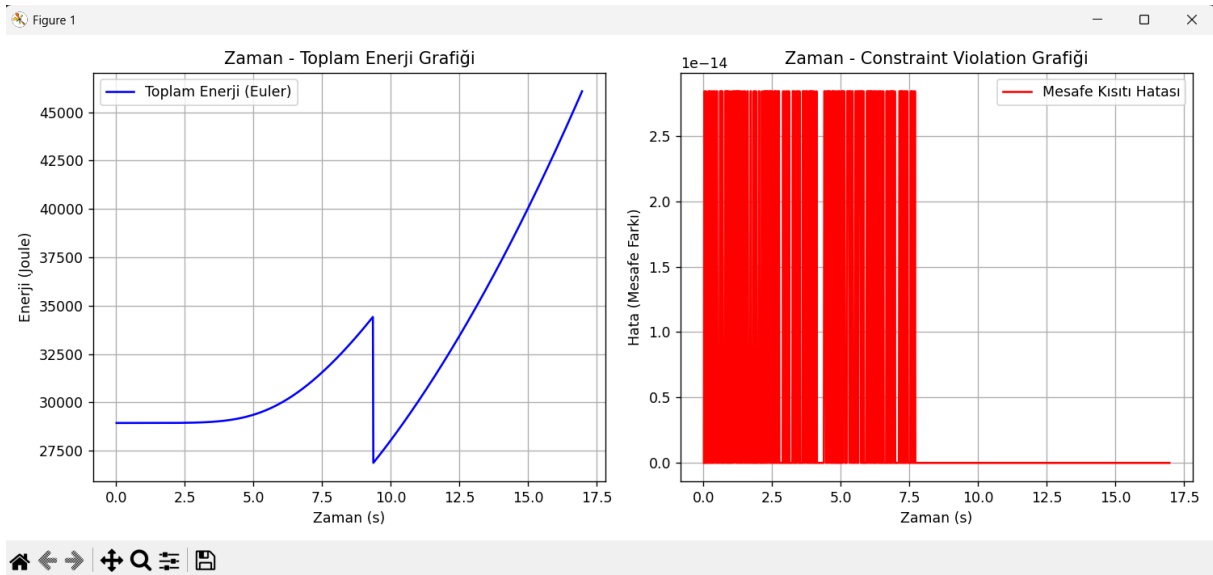
Şekil 3: Semi-Implicit Euler yöntemi kullanılarak çalıştırılan simülasyon anı. Sol üstteki bilgi ekranında görüldüğü üzere, Euler yönteminin matematiksel doğası gereği sistemin toplam enerjisinde zamanla belirgin bir sapma (yaklaşık %6.8 oranında Enerji Drifti) meydana gelmektedir.

5.2. Analiz Grafikleri

Simülasyon sonucunda üretilen enerji drifti ve kısıt ihlali (constraint violation) grafikleri aşağıda sunulmuştur:



Şekil 4: Simülasyon boyunca sistemin toplam mekanik enerjisinin ve mesafe kısıtı (constraint) hata miktarının zamana bağlı değişimi. Enerji grafiğindeki ani düşüşler zemin çarpışmaları sırasındaki enerji sönümlemesini (restitution) gösterirken; hata grafiğindeki $1e-14$ ölçeği, yazılan iteratif kısıt çözücünün kusursuz bir hassasiyetle çalıştığını kanıtlamaktadır.



Şekil 5: Semi-Implicit Euler integrasyon yöntemi kullanılarak elde edilen enerji ve hata analizi grafikleri. Şekil 4'te sunulan Verlet yöntemi grafiği ile karşılaştırıldığında, Euler yönteminin matematiksel karakteristiğinden kaynaklanan şiddetli ve sürekli bir enerji artışı (drift) açıkça görülmektedir.

6. Tartışma

Geliştirilen prototip üzerinden yapılan testler sonucunda farklı sayısal integrasyon yöntemlerinin simülasyon kararlılığı üzerindeki etkileri açıkça görülmüştür.

Enerji Kararlılığı Analizi:

Elde edilen analiz grafiklerine bakıldığında, Semi-Implicit Euler yöntemi kullanıldığında toplam enerjide zamanla kademeli bir sapma (enerji drifti) olduğu gözlemlenmiştir. Euler yöntemi, diferansiyel denklemleri birinci mertebeden yaklaşımla çözdüğü için sistemin faz uzayı hacmini genişleterek sisteme yapay enerji ekleme eğilimi göstermiştir.

Buna karşılık, Velocity Verlet yöntemi kullanıldığında sistemin toplam enerjisinin çok daha dar bir tolerans bandında kaldığı (yüksek enerji korunumu) tespit edilmiştir. Verlet yönteminin ivmeyi ve hızı ikinci mertebe doğrulukla hesaplaması, uzun süreli simülasyonlarda sistemin fiziksel gerçeklikten kopmadan (patlamadan) çalışmasını sağlamıştır.

Çarpışma ve Kısıt Çözümü Analizi:

Çarpışma durumlarında yalnızca cisimlerin hız yönlerini tersine çevirmenin yeterli olmadığı görülmüştür. Ayrık zaman adımlarından dolayı cisimler zemin veya birbirleri ile iç içe geçebilmektedir (penetrasyon). Bu durum, çarpışma impulsundan hemen önce "Pozisyon Düzeltmesi" uygulanarak başarılı bir şekilde çözülmüş ve "batma (sinking)" etkisi engellenmiştir.

Sarkaç sisteminde uygulanan mesafe kısıtında ise, kısıt hataları (violation) gözlemlenmiş ancak yazılan iteratif çözücü (Constraint Solver) sayesinde bu hata payı hızla sönümlenerek cismin hedef uzaklıkta kalması sağlanmıştır. İterasyon sayısının (deneyde 3 olarak belirlenmiştir) sistemin sertliğini (stiffness) doğrudan etkilediği görülmüştür.

7. Kaynakça

Grafik Programlama Dersi, "Basit 2D Fizik Motoru Geliştirme: Çarpışma, Constraint ve Stabilité Analizi" Vize Ödevi Dokümanı, 2024.

Catto, E. (2006). Fast and Simple Physics using Sequential Impulses. Game Developers Conference (GDC).

Eberly, D. H. (2010). Game Physics. CRC Press.